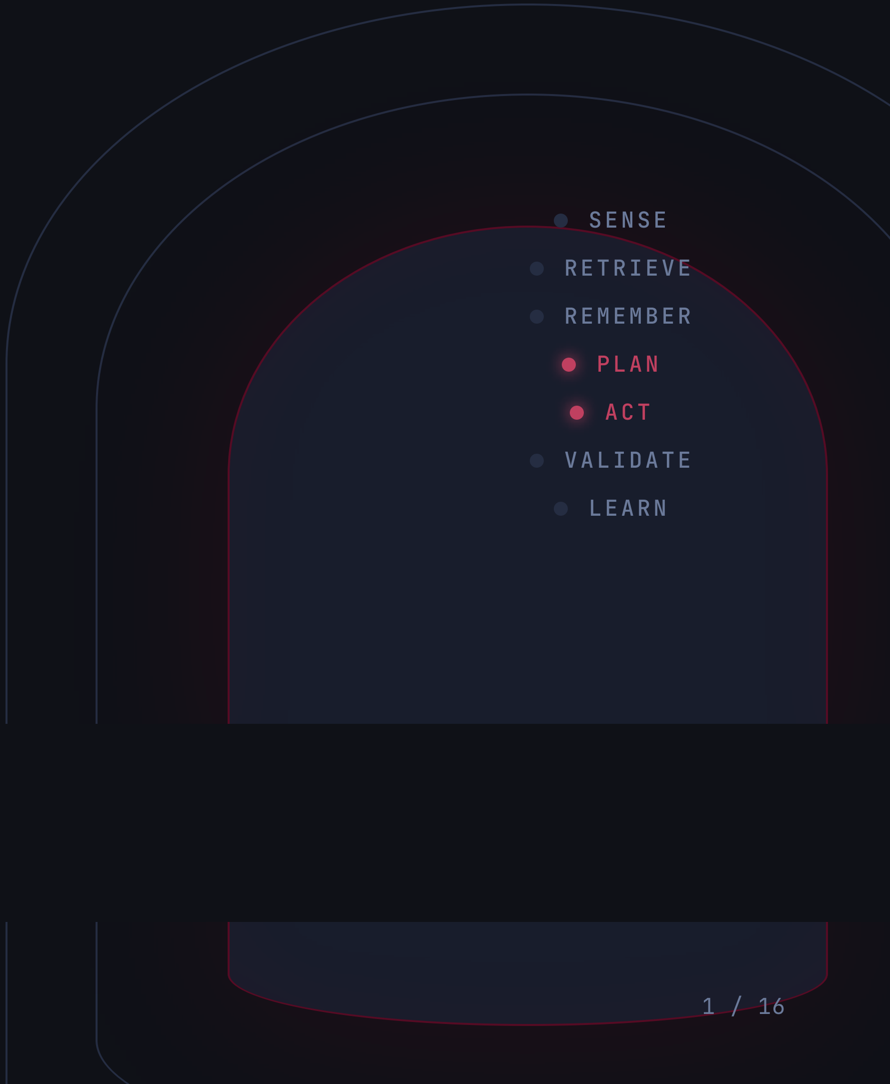
CLEW

AGENTIC DEVELOPMENT · 2026

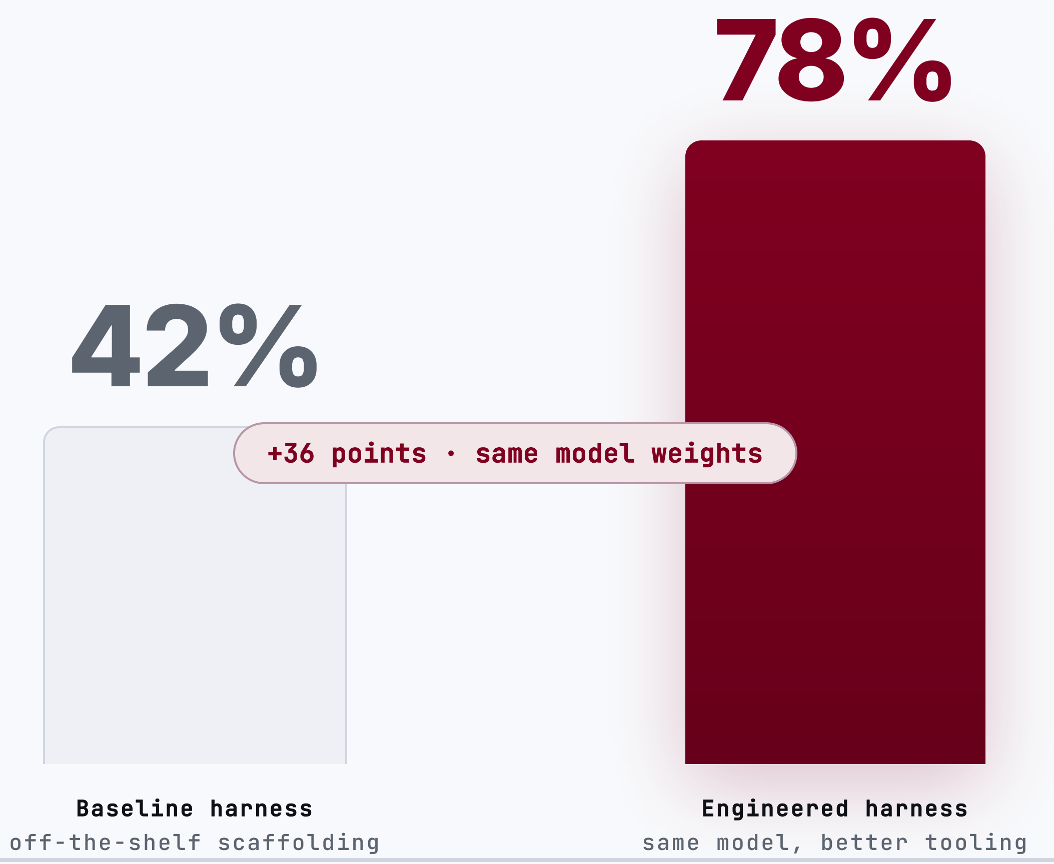
The model is the engine. The **harness** is the car.

What makes a good agentic coding harness — and how clew keeps it coherent.



THE LEVER · PROOF

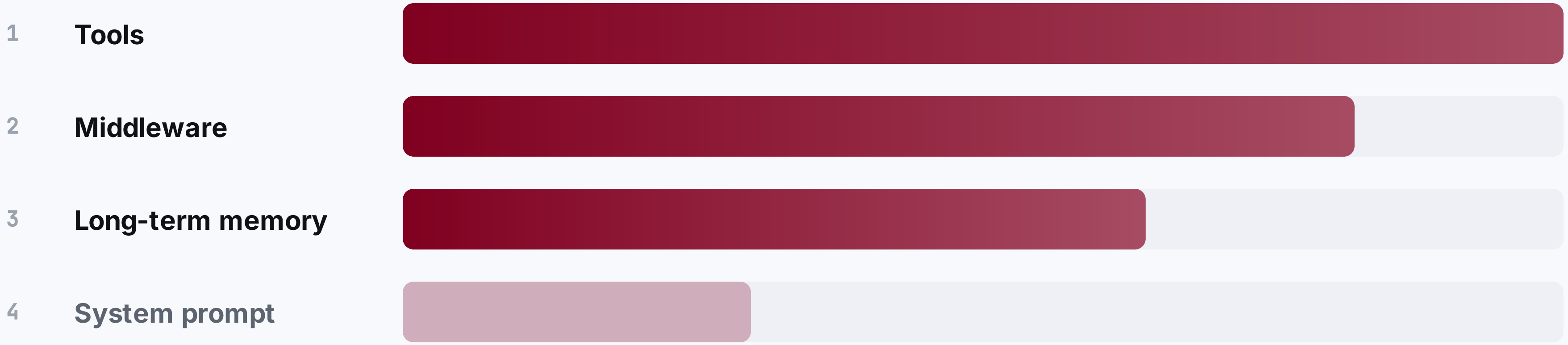
Hold the model fixed, swap the harness: 42% → 78%.



Claude Opus 4.5 · HAL CORE-Bench · the only thing that changed was the harness

Performance lives in the tooling around the model — not the model.

Ablation studies rank what actually moves agent performance. Model prompt-tuning sits **last**.



THE INVERSION

Most teams pour effort into #4 — the prompt. The gains sit in #1–3.

THE MODEL

Table stakes once it is capable. The harness is the differentiator.

HARNESS ANATOMY

One loop, 15 capabilities — a capability system, not a layer cake.

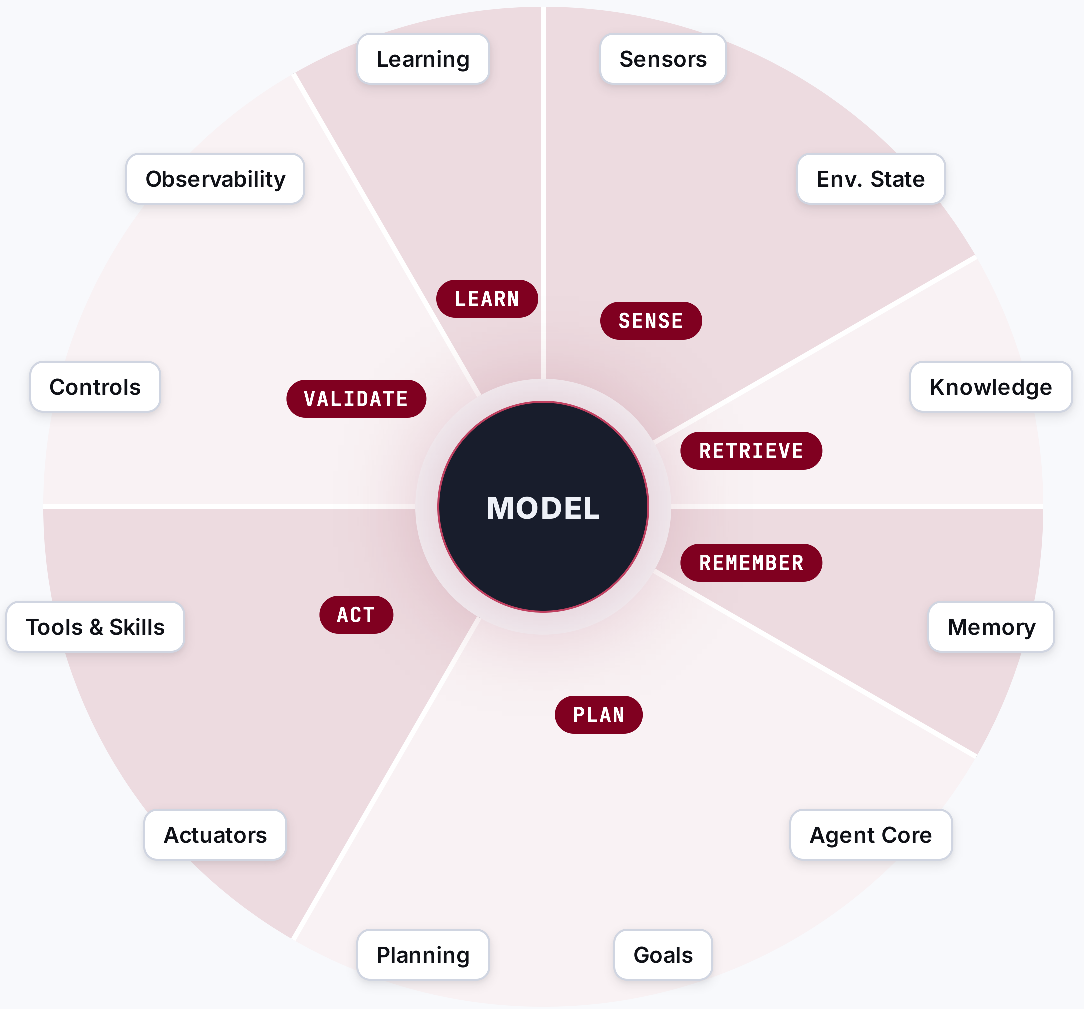
The model fills each step; the harness governs the transitions — and three capabilities govern them all.

SENSE	RETRIEVE	REMEMBER	PLAN	ACT	VALIDATE	LEARN
Sensors / Perception	Knowledge	Memory	Agent Core	Actuators / Actions	Controls & Validation	Learning & Adaptation
Environment State			Goals & Tasking	Tools & Skills	Observability	
			Planning & Decomp.			

GOVERNS EVERY STEP

Safety & Governance | Identity & Access | Human Feedback

The model is a dot. The harness is the mass around it.



CROSS-CUTTING

Govern every step — they don't belong to one.

- Safety & Governance
- Identity & Access
- Human Feedback

Each wedge is **one loop step** and the capabilities it owns · the three on the right **govern every step**.

THE GAP

Most harnesses only have the shallowest memory — yet memory is the moat.

Working memory
session scratchpad

← most stop here

Episodic memory
resumable transcript

Long-term memory
AGENTS.md · CLAUDE.md · the artefact store

the moat

Persists

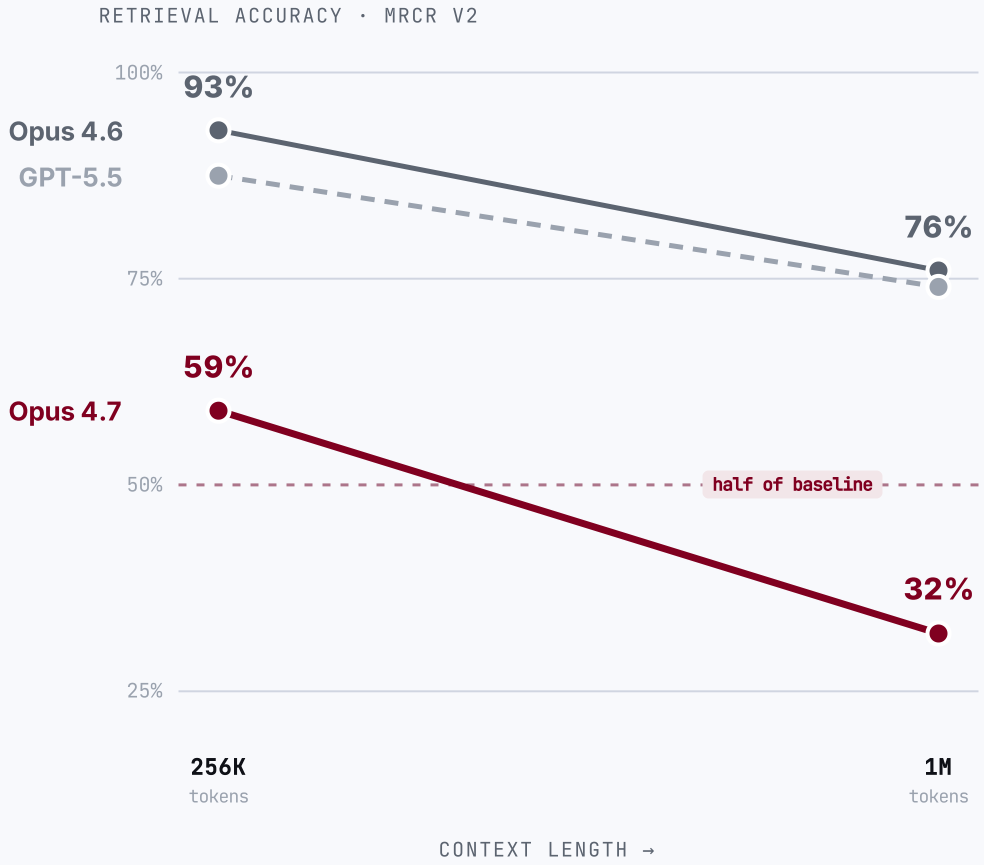
Working memory dies at session end. Long-term memory is the only layer that survives across sessions — and compounds as the project grows.

0 tokens / fetch

And it's a local file read — no MCP API roundtrip, no token cost, reloaded at the start of every session.

A bigger context window is not bigger memory.

Retrieval accuracy **rots** as you fill the window — even the 2026 frontier.



The advertised window is **not** the usable window.

59% → 32%

Claude Opus 4.7 loses ~half its multi-needle retrieval from 256K to 1M tokens. Opus 4.6 holds better (93% → 76%).

Codex too

the GPT-5.x line behind Codex CLI drops ~87% → 74% at long context — a proxy; no per-length figure is published for the Codex-tuned model.

11 of 13

models fell below half their accuracy by 32K tokens — context rot isn't new (NoLiMa, 2025).

clew engineers the deepest memory layer — so it stops rotting.

WITHOUT STRUCTURE, MEMORY ROTS

No traceability graph from the docs
No reliable view of how artefacts relate — joins across them are impossible.

No determinism if you trust the LLM
It silently renumbers IDs; you end up re-verifying everything by hand.

Refactors cost a fortune
Change one ID or name and the effort to update every reference is massive.

CLEW = STRUCTURED LONG-TERM MEMORY

End-to-end traceability
a typed graph generated from the docs themselves

Write-time integrity
deterministic IDs · FK checks · drift detection

Zero token cost per fetch
local file reads · the repo is the source of truth

From context to code — clew maps the whole SDLC.

Nineteen artefact types across three phases — discovery feeds design, design feeds delivery, all traceable.

1

Discover & Define

make the context real

Business Domain

Brainstorm, interview and run workshops with the business; model the business architecture and define the domain language.

11 artefacts

VISION · Persona · Capability · Value stream · Objective · Bounded context · Glossary · Domain model …

2

Design the Solution

fed by the context

Product specs Architecture

Decide the architecture, define contracts, break functionality down and specify requirements & quality — all tracing back to the domain.

7 artefacts

FBS · Use case · Quality attribute · PRD · Interface contract · CLI contract · ADR

3

Execute & Deliver

plan, build, test, ship

Execution

Sequence the roadmap, plan atomic increments, build and validate, then deploy and operate — covering the rest of the SDLC.

6 built · 1 planned

Delivery roadmap · Implementation plan · Getting started · Deployment · Runbook · Incident analysis · Test strategy

CROSS-CUTTING Ideas research workshop competitive landscape

discovery & enrichment that feed every phase

Discover & Define — make the context real.

WHAT THE AGENTIC ENGINEER DOES HERE

Brainstorm & capture
ideas, hypotheses

Research & scan
market, competitors

Interview & workshop
with the business

Model the business
vision → objectives

Define the domain
ubiquitous language

Business
STEPS 0–6

Vision
north star

Persona
who we serve

Lean Canvas
value proposition

Capability map
what it can do

Value stream
how value flows

Objective
goals & key results

Business process
operations

Quantitative model
unit economics

Domain
STEPS 2B–7B

Bounded context
consistency boundary

Glossary
ubiquitous language

Domain model
aggregates & events

Design the Solution — turn context into a solution.

WHAT THE AGENTIC ENGINEER DOES HERE

Decide architecture
ADRs, trade-offs

Define contracts
interface, CLI

Break down function
FBS

Specify & qualify
use cases, quality

Write PRDs
one per epic

Product specs
STEPS 7–10

Functional breakdown
feature tree

Use case
actor scenarios

Quality attribute
ISO 25010

Product requirements
one per epic

Architecture
STEPS 7C · 8.5

Interface contract
API & events

CLI contract
command surface

Decision record
architecture choices

Execute & Deliver — plan, build, test, ship.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Sequence the roadmap** — epics, phases
- **Plan increments** — atomic steps
- **Build & validate** — tests, checks
- **Deploy** — infra, release
- **Operate & fix** — runbooks, RCA

Plan & build
EXISTS

- **Delivery roadmap** — epics & phases
- **Implementation plan** — atomic increments

Validate
PLANNED · NEW QA- CATEGORY

- **Test strategy** — the one gap

Deploy & operate
EXISTS

- **Deployment view** — infra & topology
- **Getting started** — onboarding guide
- **Runbook** — operations
- **Incident analysis** — bug RCA

fed by ← Design

HOW IT WORKS

IDs minted by the app, links checked at write time — never by the LLM.

1 · AGENT ASKS

Call clew via Bash

```
clew new prd --epic E-01
```

The agent never invents an ID — it requests one.

2 · CLEW ENFORCES

Validate & mint

```
E-01 exists? ✓
FK enforced · → PRD-0001
```

Type-checked against the relationship registry; collisions rejected.

3 · AGENT WRITES

Reference the ID

```
[PRD-0001](prd-0001.md)
```

Markdown prose cites the stable ID clew returned.

✓ Deterministic — clew mints every ID and checks every link

✗ Guessed — the LLM renumbers silently and references rot

THE PAYOFF

Ask the graph anything: trace upstream, measure blast radius, prove integrity.

clew trace OBJ-02

```
OBJ-02
↑ E-01 Epic
↑ C1.2 Capability
↑ P-01 Persona
↑ VISION
```

Upstream chain — why this exists.

clew impact E-03

```
E-03 changes →
↓ PRD-0007
↓ QA-SC02
↓ Plan-0007
3 artefacts affected
```

Downstream blast radius — what breaks.

clew check

```
scanning references…
bindings ✓
drift ✓
orphans ✓
0 broken references
```

Integrity audit — deterministic, repeatable.

100% current Every internal reference is up to date, **all the time** — the north-star metric is end-to-end traceability, not work-in-progress.

THE TAKEAWAY

Engineer the harness, not just the prompt.

Start with the highest-leverage capability: memory that doesn't rot.

Not a PM tool | Not a wiki | Not a SaaS | The repo is the source of truth

Git for product architecture — manage what must be true, not what's in progress.

APPENDIX

The full capability map — 15 domains around the agent loop.

1 · Agent Core
reasoning · planning · tool select

2 · Memory
working · episodic · long-term

3 · Knowledge
repo · docs · RAG

4 · Sensors / Perception
files · runtime · telemetry

5 · Actuators / Actions
edit · execute · git

6 · Controls & Validation
tests · types · evals

7 · Observability
traces · logs · metrics

8 · Goals & Tasking
intent · success · scope

9 · Planning & Decomp.
decompose · sequence · retries

10 · Identity & Access
auth · roles · secrets

11 · Human Feedback
review · approval · handoff

12 · Learning & Adaptation
feedback · heuristics · eval-driven

13 · Safety & Governance
permissions · sandbox · policy

14 · Environment State
repo · runtime · deps

15 · Tools & Skills
shell · MCP · skills